

webapp

Eine Anwendung zur Verwaltung von Expertennetzwerken sowie Projekten und Challenges auf kommunaler Ebene. Zum Aufsetzen der Anwendung müssen folgende Schritte befolgt werden.

Voraussetzungen

Stelle sicher, dass du **Node.js** (v18.x oder höher) und einen Paketmanager (**npm**, **yarn** oder **pnpm**) installiert hast.

Installation

1. Repositorium klonen: ```bash git clone https://github.com/FischkopDev/heiProb`
2. Dependencies installieren: `npm install`
3. Anwendung testen `npm run test`
4. Server starten `npm run dev`

webapp

Modules

- [app/api/challenges/add/route](#)
- [app/api/challenges/delete/route](#)
- [app/api/challenges/list/route](#)
- [app/api/sandbox/add/route](#)
- [app/api/sandbox/list/route](#)
- [app/api/sandbox/update/route](#)
- [app/api/users/create/route](#)
- [app/api/users/delete/route](#)
- [app/api/users/list/route](#)
- [app/api/users/update/route](#)
- [app/challenge/add/page](#)
- [app/challenge/page](#)
- [app/components/Agentbar](#)
- [app/components/Sidebar](#)
- [app/components/Topbar](#)
- [app/layout](#)
- [app/page](#)
- [app/relations/add/page](#)
- [app/relations/page](#)
- [app/relations/Person](#)
- [app/relations/update/page](#)
- [app/sandbox/add/page](#)
- [app/sandbox/page](#)
- [app/sandbox/project/details/page](#)
- [app/search/page](#)
- [lib/db](#)
- [lib/types](#)

lib/types

Interfaces

- [AddExpertViewProps](#)
- [Challenge](#)
- [Expert](#)
- [ExpertenNetzwerkViewProps](#)
- [ExpertFormData](#)
- [ExpertOption](#)
- [NewProject](#)
- [Organization](#)
- [ProblemItem](#)
- [Project](#)
- [ProjectDetails](#)
- [ProjectMember](#)

Interface: AddExpertViewProps

Defined in: [lib/types.ts:270](#)

Eigenschaften (Props) für eine Komponente zum Hinzufügen oder Bearbeiten eines Experten. Callbacks sind optional, da die Komponente alternativ integrierte Fallback-Navigation besitzt. * mermaid classDiagram direction LR

AddExpertViewProps ..> ExpertFormData : nutzt class

```
AddExpertViewProps { +onSave(formData: ExpertFormData) void  
+onCancel() void }
```

Properties

onCancel?

optional **onCancel?**: () => void

Defined in: [lib/types.ts:279](#)

Callback-Funktion, um den Vorgang abubrechen und zur vorherigen Ansicht zurückzukehren.

Returns

void

onSave?

optional **onSave?**: (formData) => void

Defined in: [lib/types.ts:275](#)

Callback-Funktion, die aufgerufen wird, wenn die Expertendaten erfolgreich validiert und gespeichert wurden.

Parameters

`formData`

[ExpertFormData](#)

Die eingegebenen Formulardaten des Experten.

Returns

`void`

Interface: Challenge

Defined in: [lib/types.ts:13](#)

Repräsentiert eine städtische oder organisatorische Herausforderung (Challenge). *

```
mermaid classDiagram class Challenge { +string id +string title +string department +string status }
```

Properties

department

department: string

Defined in: [lib/types.ts:19](#)

Das zuständige Amt oder die Abteilung.

id

id: string

Defined in: [lib/types.ts:15](#)

Eindeutige ID der Challenge.

status

status: string

Defined in: [lib/types.ts:21](#)

Der aktuelle Status der Challenge (z. B. "Aktiv", "In Vorbereitung").

title

title: string

Defined in: [lib/types.ts:17](#)

Der Titel oder die Überschrift der Challenge.

Interface: Expert

Defined in: [lib/types.ts:68](#)

Repräsentiert ein Profil einer Expert*in oder einer Ansprechperson. * mermaid
classDiagram class Expert { +string id +string name +string role
+string[] skills }

Properties

id

id: string

Defined in: [lib/types.ts:70](#)

Eindeutige ID der Expert*in.

name

name: string

Defined in: [lib/types.ts:72](#)

Der vollständige Name.

role

role: string

Defined in: [lib/types.ts:74](#)

Die fachliche Rolle oder Kernkompetenz.

skills

skills: string[]

Defined in: [lib/types.ts:76](#)

Eine Liste von spezifischen Fähigkeiten oder Schlagworten (Skills).

Interface: ExpertFormData

Defined in: [lib/types.ts:304](#)

Formulardaten-Struktur für Expert*innen. * mermaid classDiagram class ExpertFormData { +string name +string prename +string title +string primary_organization +string other_organizations +string scientificAreas +string email +string phone +string last_contact +string description +string expert_fields +boolean economic +boolean science +boolean social }

Properties

description

description: string

Defined in: [lib/types.ts:324](#)

Eine detaillierte Beschreibung oder Kurzbiografie des Experten.

economic

economic: boolean

Defined in: [lib/types.ts:328](#)

Indikator, ob der Experte im wirtschaftlichen Bereich tätig ist.

email

email: string

Defined in: [lib/types.ts:318](#)

Die E-Mail-Adresse des Experten.

expert_fields

expert_fields: string

Defined in: [lib/types.ts:326](#)

Spezifische Fachbereiche oder Felder, in denen der Experte tätig ist.

last_contact?

optional **last_contact?:** string

Defined in: [lib/types.ts:322](#)

Datum des letzten Kontakts (optional).

name

name: string

Defined in: [lib/types.ts:306](#)

Der Nachname des Experten.

other_organizations

other_organizations: string

Defined in: [lib/types.ts:314](#)

Weitere Zugehörigkeiten oder Organisationen (kommagetrennt).

phone

phone: string

Defined in: [lib/types.ts:320](#)

Die Telefonnummer des Experten.

prename

prename: string

Defined in: [lib/types.ts:308](#)

Der Vorname des Experten.

primary_organization

primary_organization: string

Defined in: [lib/types.ts:312](#)

Die primäre Zugehörigkeit oder Organisation.

science

science: boolean

Defined in: [lib/types.ts:330](#)

Indikator, ob der Experte im wissenschaftlichen Bereich tätig ist.

scientificAreas

scientificAreas: string

Defined in: [lib/types.ts:316](#)

Wissenschaftliche oder fachliche Arbeitsbereiche.

social

social: boolean

Defined in: [lib/types.ts:332](#)

Indikator, ob der Experte im sozialen Bereich tätig ist.

title

title: string

Defined in: [lib/types.ts:310](#)

Der Titel oder akademische Grad (z. B. "Prof. Dr.).

Interface: ExpertOption

Defined in: [lib/types.ts:164](#)

Struktur für die Auswahlliste (Dropdown) der verfügbaren Experten. * mermaid
classDiagram class ExpertOption { +id: number | string +string
name }

Properties

id

id: string | number

Defined in: [lib/types.ts:166](#)

Die ID des Experten (as number or string).

name

name: string

Defined in: [lib/types.ts:168](#)

Der vollständige Name des Experten.

Interface: ExpertenNetzwerkViewProps

Defined in: [lib/types.ts:344](#)

Eigenschaften (Props) für die Komponente, die eine Netzwerk-/Beziehungsübersicht anzeigt.*

```
mermaid classDiagram class ExpertenNetzwerkViewProps { +unknown key }
```

Indexable

[key: string]: unknown

Versionsinformation oder zusätzliche Kontextdaten (optional).

Interface: NewProject

Defined in: [lib/types.ts:190](#)

Struktur für das State-Objekt eines neu anzulegenden Projekts. * mermaid

```
classDiagram direction LR NewProject "1" --* "many"
```

```
ProjectMember : enthält class NewProject { +string title +string  
description +string startDate +string endDate +string state  
+string location +string websiteUrl +string details  
+ProjectMember[] members }
```

Properties

description

description: string

Defined in: [lib/types.ts:194](#)

Eine kurze Zusammenfassung oder Beschreibung des Projekts.

details

details: string

Defined in: [lib/types.ts:206](#)

Zusätzliche, detaillierte Projektinformationen.

endDate

endDate: string

Defined in: [lib/types.ts:198](#)

Das Enddatum des Projekts (Format: YYYY-MM-DD).

location

location: string

Defined in: [lib/types.ts:202](#)

Der geografische oder organisatorische Ort des Projekts.

members

members: [ProjectMember](#)[]

Defined in: [lib/types.ts:208](#)

Liste der dem Projekt zugewiesenen Mitglieder.

startDate

startDate: string

Defined in: [lib/types.ts:196](#)

Das Startdatum des Projekts (Format: YYYY-MM-DD).

state

state: string

Defined in: [lib/types.ts:200](#)

Der aktuelle Projektstatus (z. B. 'Ideen-Phase').

title

title: string

Defined in: [lib/types.ts:192](#)

Der Titel des Projekts.

websiteUrl

websiteUrl: string

Defined in: [lib/types.ts:204](#)

Optionale URL zur Projekt-Website.

Interface: Organization

Defined in: [lib/types.ts:359](#)

Repräsentiert eine Organisation. * mermaid classDiagram class Organization { +number id +string name }

Properties

id

id: number

Defined in: [lib/types.ts:361](#)

Eindeutige ID der Organisation.

name

name: string

Defined in: [lib/types.ts:363](#)

Der Name der Organisation.

Interface: ProblemItem

Defined in: [lib/types.ts:99](#)

Repräsentiert eine Herausforderung oder ein Problem im System. * mermaid
classDiagram class ProblemItem { +number id +number problem_id
+string title +string tags +string category +string status
+string statusColor +string description +string summary +string
impact +string stakeholders +string nextSteps }

Properties

category

category: string

Defined in: [lib/types.ts:109](#)

Die übergeordnete Kategorie des Problems.

description?

optional **description?:** string

Defined in: [lib/types.ts:115](#)

Eine detaillierte Beschreibung des Problems oder kurze Zusammenfassung.

id?

optional **id?:** number

Defined in: [lib/types.ts:101](#)

Eindeutige ID des Problems (Primärschlüssel).

impact?

optional **impact?**: string

Defined in: [lib/types.ts:119](#)

Die Auswirkungen oder Konsequenzen, die das Problem verursacht.

nextSteps?

optional **nextSteps?**: string

Defined in: [lib/types.ts:123](#)

Die nächsten geplanten Schritte zur Lösung des Problems.

problem_id?

optional **problem_id?**: number

Defined in: [lib/types.ts:103](#)

Alternativer Feldname für die Problem-ID.

stakeholders?

optional **stakeholders?**: string

Defined in: [lib/types.ts:121](#)

Die betroffenen Personen, Abteilungen oder Stakeholder.

status

status: "Ungelöst" | "In Bearbeitung" | "Gelöst"

Defined in: [lib/types.ts:111](#)

Der aktuelle Bearbeitungsstatus.

statusColor

statusColor: "amber" | "green" | "slate"

Defined in: [lib/types.ts:113](#)

Die dem Status zugewiesene UI-Farbe.

summary?

optional **summary?**: string

Defined in: [lib/types.ts:117](#)

Alternativer Feldname für Beschreibung.

tags?

optional **tags?**: string

Defined in: [lib/types.ts:107](#)

Kommagetrennte oder formatierte Tags/Schlagworte zur Verschlagwortung.

title

title: string

Defined in: [lib/types.ts:105](#)

Der Titel oder die Überschrift des Problems.

Interface: Project

Defined in: [lib/types.ts:39](#)

Repräsentiert ein registriertes (Smart-City-)Projekt. * mermaid classDiagram class Project { +string id +string title +string stage +string topics +string location +string[] actors +string value }

Properties

actors?

optional **actors?**: string[]

Defined in: [lib/types.ts:51](#)

Eine Liste der Namen aller beteiligten Akteure/Experten.

id

id: string

Defined in: [lib/types.ts:41](#)

Eindeutige ID des Projekts.

location?

optional **location?**: string

Defined in: [lib/types.ts:49](#)

Der geografische Ort oder Stadtteil des Projekts.

stage?

optional **stage?**: string

Defined in: [lib/types.ts:45](#)

Die aktuelle Projektphase oder Themen (z. B. "Test-Phase", "Technik").

title

title: string

Defined in: [lib/types.ts:43](#)

Der Projekttitel.

topics?

optional **topics?**: string

Defined in: [lib/types.ts:47](#)

Alternativer Feldname für Themen oder Standort.

value?

optional **value?**: string

Defined in: [lib/types.ts:53](#)

Ein dynamischer UI-Anzeigewert (Teaser-Text, URL oder formatiertes Update-Datum).

Interface: ProjectDetails

Defined in: [lib/types.ts:232](#)

Struktur für die vollständigen Details eines Projekts. * mermaid classDiagram
direction LR ProjectDetails "1" --* "many" ProjectMember :
besitzt class ProjectDetails { +string id +string title +string
description +string startDate +string endDate +string state
+string project_state +string location +string websiteUrl +string
details +ProjectMember[] experts }

Properties

description

description: string

Defined in: [lib/types.ts:238](#)

Kurze Zusammenfassung oder Beschreibung des Projekts.

details

details: string

Defined in: [lib/types.ts:252](#)

Zusätzliche, detaillierte Projektbeschreibungen oder Notizen.

endDate

endDate: string

Defined in: [lib/types.ts:242](#)

Enddatum des Projekts (Format: YYYY-MM-DD).

experts

experts: [ProjectMember](#)[]

Defined in: [lib/types.ts:254](#)

Liste aller dem Projekt zugewiesenen Experten.

id

id: string

Defined in: [lib/types.ts:234](#)

Eindeutige ID des Projekts.

location

location: string

Defined in: [lib/types.ts:248](#)

Geografischer oder organisatorischer Standort des Projekts.

project_state?

optional project_state?: string

Defined in: [lib/types.ts:246](#)

Alternativer oder datenbankspezifischer Projektstatus.

startDate

startDate: string

Defined in: [lib/types.ts:240](#)

Startdatum des Projekts (Format: YYYY-MM-DD).

state

state: string

Defined in: [lib/types.ts:244](#)

Der aktuelle Projektstatus (z. B. 'Ideen-Phase').

title

title: string

Defined in: [lib/types.ts:236](#)

Der Projekttitel.

websiteUrl

websiteUrl: string

Defined in: [lib/types.ts:250](#)

Optionale Projekt-Website-URL.

Interface: ProjectMember

Defined in: [lib/types.ts:138](#)

Repräsentiert ein Mitglied innerhalb eines Projekts inklusive seiner RACI-Rolle. *

```
mermaid classDiagram class ProjectMember { +string id +number expertId +string name +string role }
```

Properties

expertId?

optional **expertId?**: number

Defined in: [lib/types.ts:142](#)

Die echte ID des Experten aus der Datenbank (optional).

id

id: string

Defined in: [lib/types.ts:140](#)

Eindeutige temporäre ID für das UI-Mapping (z. B. generiert über `Date.now()`).

name

name: string

Defined in: [lib/types.ts:144](#)

Der Name des Experten.

role

role: "R" | "A" | "C" | "I"

Defined in: [lib/types.ts:151](#)

Die RACI-Rolle des Mitglieds im Projekt: - R: Responsible (Durchführungsverantwortlich)
- A: Accountable (Kosten-/Gesamtverantwortlich) - C: Consulted (Fachlich beratend) - I:
Informed (Zu informieren)

[webapp](#)

[webapp](#) / lib/db

lib/db

Variables

- [default](#)

[webapp](#)

[webapp](#) / [lib/db](#) / default

Variable: default

```
const default: Pool
```

Defined in: [lib/db.ts:3](#)

[webapp](#)

[webapp](#) / app/api/users/update/route

app/api/users/update/route

Functions

- [addOrganization](#)
- [getOrganizationIdByName](#)
- [PATCH](#)
- [updateExpert](#)

Function: PATCH()

```
PATCH(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ data: any; success: boolean; }>>
```

Defined in: [app/api/users/update/route.ts:173](#)

Handler für HTTP PATCH-Anfragen zum Aktualisieren eines bestehenden Experten.

Extrahiert die `expert_id` aus dem Request-Body, trennt sie von den Update-Daten und delegiert die Aktualisierung an `updateExpert()`.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

```
Promise<NextResponse<{ error: string; }> | NextResponse<{ data: any;  
success: boolean; }>>
```

Ein JSON-Antwortobjekt: - 200: Erfolg mit dem aktualisierten Experten-Objekt in `data`. - 400: Fehlende `expert_id`. - 404: Kein Experte mit der angegebenen ID gefunden. - 500: Interner Serverfehler bei der Datenbankabfrage.

Example

```
// Erwarteter Request-Body:  
{
```

```
"expert_id": 123,  
"title": "Dr.",  
"primary_organization": "Neue Firma AG"  
}
```

Remarks

- Nur die im Body enthaltenen Felder werden aktualisiert; alle anderen bleiben unverändert.
- Die Fehlermeldung bei fehlender ID ist auf Englisch, während andere Teile des Codes teils Deutsch verwenden (Konsistenz könnte verbessert werden).

Function: addOrganization()

```
addOrganization(name, location, field, description):  
Promise<any>
```

Defined in: [app/api/users/update/route.ts:19](#)

Fügt eine neue Organisation in die Datenbank ein.

Parameters

name

string

Der Name der Organisation (erforderlich).

location

string

Der Standort der Organisation.

field

string

Das Tätigkeitsfeld der Organisation.

description

string

Eine Beschreibung der Organisation.

Returns

Promise<any>

Die neu generierte `organization_id` bei Erfolg. Gibt 0 zurück, wenn ein Fehler auftritt oder kein ID-Wert zurückgegeben wird.

Remarks

- Logs Fehler an die Konsole, wirft aber keine Exception, um den Aufrufer nicht zu blockieren.
- Die Rückgabe von 0 bei Fehlern erfordert eine explizite Prüfung durch den Aufrufer.

Function: `getOrganizationIdByName()`

`getOrganizationIdByName(name): Promise<number | null>`

Defined in: [app/api/users/update/route.ts:48](#)

Sucht die eindeutige ID einer Organisation anhand ihres Namens.

Parameters

name

string

Der Name der Organisation, nach dem gesucht werden soll.

Returns

Promise<number | null>

Die `organization_id` als Zahl, falls gefunden, sonst `null`.

Remarks

- Nutzt `LIMIT 1`, da von eindeutigen Namen ausgegangen wird.
- Fängt Datenbankfehler ab und gibt `null` zurück.

Function: updateExpert()

updateExpert(id, body): Promise<any>

Defined in: [app/api/users/update/route.ts:99](#)

Aktualisiert die Daten eines Experten in der Datenbank.

Dieser Prozess beinhaltet: 1. Prüfung und ggf. automatische Erstellung der angegebenen `primary_organization`. 2. Aktualisierung der Expertenfelder unter Verwendung von `COALESCE`, um nicht übergebene Werte beizubehalten.

Parameters

id

number

Die ID des zu aktualisierenden Experten (`expert_id`).

body

any

Das Objekt mit den zu aktualisierenden Daten.

Returns

Promise<any>

Das aktualisierte Experten-Objekt (inkl. aller Felder) bei Erfolg. Gibt `undefined` zurück, wenn kein Experte mit der angegebenen ID existiert.

Remarks

- Wenn `primary_organization` angegeben ist, wird geprüft, ob sie existiert. Falls nicht, wird sie mit dem aktuellen `location` (falls vorhanden) und leeren Feldern für `field/description` erstellt.
- Die SQL-Abfrage nutzt `COALESCE($1, column)`, um nur übermittelte Werte zu überschreiben.
- Falls `primary_organization` nicht im Body ist, bleibt die bisherige Zuordnung erhalten.

[webapp](#)

[webapp](#) / app/api/users/create/route

app/api/users/create/route

Functions

- [addExpert](#)
- [addExpertFields](#)
- [addOrganization](#)
- [getOrganizationIdByName](#)
- [POST](#)

Function: POST()

```
POST(request): Promise<NextResponse<{ error: string;
required: string[]; }> | NextResponse<{ expertId: any;
fieldsAdded: number; success: boolean; }> | NextResponse<{
details: any; error: string; }>>
```

Defined in: [app/api/users/create/route.ts:195](#)

Handler für HTTP POST-Anfragen zum Erstellen eines neuen Experten.

Validiert die erforderlichen Felder (name, prename, email) und delegiert die Erstellung an `addExpert()`.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

```
Promise<NextResponse<{ error: string; required: string[]; }> |
NextResponse<{ expertId: any; fieldsAdded: number; success: boolean; }> |
NextResponse<{ details: any; error: string; }>>
```

Ein JSON-Antwortobjekt: - 200: Erfolg mit { success: true, expertId: number, fieldsAdded: number } - 400: Fehlende Pflichtfelder - 500: Interner Serverfehler

Example

```
// Erwartetes Body-Format:  
{  
  "name": "Max",  
  "prename": "Mustermann",  
  "email": "max@example.com",  
  "primary_organization": "Beispiel GmbH",  
  "title": "Dr.",  
  "location": "Berlin",  
  "economic": true,  
  "expertFields": ["KI", "Machine Learning", "Data Science"]  
}
```

Function: addExpert()

```
addExpert(body): Promise<{ expertId: any; fields:
  QueryResult<any>[]; }>
```

Defined in: [app/api/users/create/route.ts:139](#)

Erstellt einen neuen Experten in der Datenbank.

Dieser Prozess beinhaltet: 1. Prüfung, ob die angegebene `primary_organization` existiert. 2. Falls nein: Automatische Erstellung der Organisation mit Standardwerten für `field` und `description`. 3. Einfügen des Experten mit der aufgelösten Organisations-ID. 4. Optional: Einfügen der Expertisefelder (`expertFields`).

Parameters

body

any

Das Objekt mit den Experten-Daten.

Returns

```
Promise<{ expertId: any; fields: QueryResult<any>[]; }>
```

Objekt mit `expertId` und `fields` (Array von Einfügergebnissen).

Remarks

- Die Funktion erstellt automatisch eine Organisation, wenn sie nicht existiert, wobei `field` und `description` leer gesetzt werden.
- Boolesche Felder (`economic`, `science`, `social`) defaults auf `false`, wenn nicht übergeben.

- Expertisefelder werden nach dem Einfügen des Experten hinzugefügt.

Function: addExpertFields()

```
addExpertFields(expertId, expertFields):  
  Promise<QueryResult<any>[]>
```

Defined in: [app/api/users/create/route.ts:82](#)

Fügt Expertisefelder für einen Experten in die Datenbank ein.

Parameters

expertId

number

Die ID des Experten.

expertFields

string[]

Array von Feld-Strings, die die Expertisen des Experten darstellen.

Returns

Promise<QueryResult<any>[]>

Array der Datenbank-Einfügeergebnisse (pg.QueryResult[]).

Remarks

- Fängt Fehler ab und loggt sie, wirft aber keine Exception.
- Gibt ein leeres Array zurück, wenn expertFields leer oder undefined ist.

Function: addOrganization()

```
addOrganization(name, location, field, description):  
  Promise<any>
```

Defined in: [app/api/users/create/route.ts:19](#)

Fügt eine neue Organisation in die Datenbank ein.

Parameters

name

string

Der Name der Organisation (erforderlich).

location

string

Der Standort der Organisation.

field

string

Das Tätigkeitsfeld der Organisation.

description

string

Eine Beschreibung der Organisation.

Returns

Promise<any>

Die neu generierte `organization_id` bei Erfolg. Gibt 0 zurück, wenn ein Fehler auftritt oder kein ID-Wert zurückgegeben wird.

Remarks

- Logs Fehler an die Konsole, wirft aber keinen Exception, um den Aufrufer nicht zu blockieren.
- Die Rückgabe von 0 bei Fehlern erfordert eine explizite Prüfung durch den Aufrufer.

Function: `getOrganizationIdByName()`

`getOrganizationIdByName(name): Promise<number | null>`

Defined in: [app/api/users/create/route.ts:48](#)

Sucht die eindeutige ID einer Organisation anhand ihres Namens.

Parameters

name

string

Der Name der Organisation, nach dem gesucht werden soll.

Returns

`Promise<number | null>`

Die `organization_id` als Zahl, falls gefunden, sonst `null`.

Remarks

- Nutzt `LIMIT 1`, da von eindeutigen Namen ausgegangen wird.
- Fängt Datenbankfehler ab und gibt `null` zurück.

[webapp](#)

[webapp](#) / app/api/users/list/route

app/api/users/list/route

Functions

- [GET](#)
- [getListOfPeopleWithOrganization](#)

Function: GET()

```
GET(): Promise<NextResponse<{ count: number; experts: any[];
  success: boolean; }> | NextResponse<{ details: any; error:
  string; }>>
```

Defined in: [app/api/users/list/route.ts:82](#)

Handler für HTTP GET-Anfragen zum Abrufen der Liste aller Personen mit Organisationsdaten und Expertisefeldern.

Delegiert die Datenbankabfrage an `getListOfPeopleWithOrganization()` und formatiert die Antwort als JSON mit Erfolgsmeldung, Ergebnisliste und Gesamtanzahl.

Returns

```
Promise<NextResponse<{ count: number; experts: any[]; success: boolean; }
> | NextResponse<{ details: any; error: string; }>>
```

Ein JSON-Antwortobjekt mit:

- `success`: Boolean, ob die Anfrage erfolgreich war
- `experts`: Array der Experten-Objekte (inkl. Organisations- und Expertisefelder-Daten)
- `count`: Anzahl der zurückgegebenen Experten

Throws

Gibt bei Fehlern eine JSON-Antwort mit Status 500 zurück und enthält Fehlerdetails im `details`-Feld.

Example

```
// Beispielantwort:
{
  "success": true,
```

```
"experts": [  
  {  
    "expert_id": 1,  
    "name": "Anna",  
    "prename": "Schmidt",  
    "organization": {  
      "organization_id": 5,  
      "name": "TechCorp",  
      "location": "Berlin",  
      "field": "IT",  
      "description": "Ein Technologieunternehmen"  
    },  
    "expertFields": ["Künstliche Intelligenz", "Machine Learning", "Dat  
  },  
],  
"count": 1  
}
```

Function:

getListOfPeopleWithOrganization()

`getListOfPeopleWithOrganization(): Promise<any[]>`

Defined in: [app/api/users/list/route.ts:21](#)

Ruft eine Liste von Experten aus der Datenbank ab, jeweils ergänzt um ihre zugehörige Organisation und Expertisefelder.

Diese Funktion führt JOINS zwischen den Tabellen "Expert", "Organization" und "ExpertField" durch, um die Organisationsdaten und Expertisefelder als verschachtelte JSON-Objekte zurückzugeben.

Returns

`Promise<any[]>`

Ein Array von Objekten, wobei jedes Objekt alle Felder des Experten (`e.*`) sowie folgende Felder enthält: - `organization`: JSON-Objekt mit Organisations-Daten (`organization_id`, `name`, `location`, `field`, `description`) - `expertFields`: JSON-Array der Expertisefelder des Experten

Remarks

- Die Ergebnisse sind alphabetisch nach dem Vornamen (`e.name`) sortiert.
- Die Abfrage ist auf maximal 50 Ergebnisse begrenzt (`LIMIT 50`).
- Nur Experten mit einer gültigen Zuordnung zu einer Organisation werden zurückgegeben (`INNER JOIN`).
- Expertisefelder werden mittels `LEFT JOIN` und Aggregation geladen, Experten ohne Felder erhalten ein leeres Array.

[webapp](#)

[webapp](#) / app/api/users/delete/route

app/api/users/delete/route

Functions

- [DELETE](#)
- [deleteExpert](#)

Function: DELETE()

```
DELETE(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ message: string; success: boolean; }>>
```

Defined in: [app/api/users/delete/route.ts:75](#)

Handler für HTTP DELETE-Anfragen zum Entfernen eines Experten.

Extrahiert die `expert_id` aus dem Request-Body, validiert sie und ruft die Löschfunktion auf. Unterscheidet zwischen fehlender ID, nicht existierendem Experten und technischen Fehlern.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

```
Promise<NextResponse<{ error: string; }> | NextResponse<{ message:  
string; success: boolean; }>>
```

Ein JSON-Antwortobjekt: - 200: Erfolg mit Bestätigungsmeldung. - 400: Fehlende oder ungültige `expert_id` im Body. - 404: Kein Experte mit der angegebenen ID gefunden. - 500: Interner Serverfehler bei der Datenbankabfrage.

Example

```
// Erwarteter Request-Body:  
{ "expert_id": 123 }
```

Remarks

- Die Validierungsmeldungen sind teilweise auf Deutsch ("User ist nicht vorhanden"), während die Erfolgsnachrichten auf Englisch sind. Für Konsistenz könnte man die Sprache vereinheitlichen.

Function: deleteExpert()

deleteExpert(id): Promise<boolean>

Defined in: [app/api/users/delete/route.ts:19](#)

Löscht einen Experten aus der Datenbank basierend auf der ID.

Parameters

id

number

Die eindeutige ID des zu löschenden Experten (expert_id).

Returns

Promise<boolean>

true, wenn ein Datensatz erfolgreich gelöscht wurde. false, wenn kein Datensatz mit dieser ID existierte (und somit nichts gelöscht wurde).

Throws

Wirft den ursprünglichen Datenbankfehler, falls die Abfrage technisch fehlschlägt (z.B. Verbindungsfehler, Syntaxfehler), damit der Aufrufer dies behandeln kann.

Remarks

- Die Abfrage nutzt RETURNING expert_id, um zu prüfen, ob tatsächlich eine Zeile betroffen war.

- Ein false-Rückgabewert bedeutet nicht zwangsläufig einen Fehler, sondern dass die ID nicht existierte.

[webapp](#)

[webapp](#) / app/api/challenges/add/route

app/api/challenges/add/route

Functions

- [addChallenge](#)
- [POST](#)

Function: POST()

```
POST(request): Promise<NextResponse<{ error: string;
  required: string[]; }> | NextResponse<{ status: number; success:
  boolean; warning: string; }> | NextResponse<{ problemId: number;
  status: number; success: boolean; }> | NextResponse<{ details:
  any; error: string; }>>
```

Defined in: [app/api/challenges/add/route.ts:80](#)

HTTP POST-Handler zum Erstellen eines neuen Challenges.

Dieser Endpunkt validiert die eingehenden Daten, prüft auf erforderliche Felder und ruft die interne Funktion addChallenge auf, um den Datensatz zu speichern.

Parameters

request

Request

Das HTTP-Request-Objekt, das den JSON-Body enthält.

Returns

```
Promise<NextResponse<{ error: string; required: string[]; }> |
NextResponse<{ status: number; success: boolean; warning: string; }> |
NextResponse<{ problemId: number; status: number; success: boolean; }> |
NextResponse<{ details: any; error: string; }>>
```

Ein NextResponse-Objekt mit folgendem Inhalt: - **200 OK**: Bei Erfolg. { success: true, status: 200 } - **400 Bad Request**: Wenn erforderliche Felder fehlen. { error: "Missing required fields", required: [...] } - **500 Internal**

Server Error: Bei Datenbankfehlern oder unerwarteten Ausnahmen. { error: "Failed to add challenge", details: error.message }

Example

```
// Beispiel für einen erfolgreichen Request
const response = await fetch('/api/challenges', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    title: "Rekursive Fibonacci",
    category: "Algorithms",
    state: "active",
    description: "Berechne die Fibonacci-Zahl..."
  })
});
const data = await response.json();
console.log(data); // { success: true, status: 200 }
```

Function: addChallenge()

addChallenge(body): Promise<number>

Defined in: [app/api/challenges/add/route.ts:21](#)

Fügt ein neues Problem (Challenge) in die Datenbank ein.

Parameters

body

Der Inhalt aus der JSON-Anfrage, der die Details des Problems enthält.

category

string

Die Kategorie des Problems.

description?

string

Eine detaillierte Beschreibung des Problems.

state

string

Der aktuelle Status/Zustand des Problems (z.B. "open", "closed").

title

string

Der Titel des Problems.

Returns

Promise\<number>

Die ID des neu erstellten Problems als Zahl. Gibt 0 zurück, falls ein Fehler auftritt oder kein ID-Wert generiert wurde.

Throws

Wirft einen Fehler, wenn die Datenbankverbindung fehlschlägt oder die Abfrage ungültig ist. Der Fehler wird jedoch im catch-Block abgefangen und loggt, gibt aber 0 zurück.

[webapp](#)

[webapp](#) / app/api/challenges/list/route

app/api/challenges/list/route

Functions

- [GET](#)
- [getListOfChallenges](#)

Function: GET()

```
GET(): Promise<NextResponse<{ challenges: any[]; count: number;
  success: boolean; }> | NextResponse<{ details: any; error:
  string; }>>
```

Defined in: [app/api/challenges/list/route.ts:63](#)

HTTP GET-Handler zum Abrufen der Liste aller Challenges.

Dieser Endpunkt ruft die Funktion `getListOfChallenges` auf und gibt die Ergebnisse als JSON-Objekt zurück. Er enthält Metadaten über den Erfolg und die Anzahl der Ergebnisse.

Returns

```
Promise<NextResponse<{ challenges: any[]; count: number; success:
  boolean; }> | NextResponse<{ details: any; error: string; }>>
```

Ein `NextResponse`-Objekt mit folgendem Inhalt: - **200 OK**: Bei Erfolg. `json { "success": true, "challenges": [ /* Array von Challenge-Objekten */ ], "count": 50 /* Anzahl der zurückgegebenen Einträge */ }` - **500 Internal Server Error**: Bei Datenbankfehlern oder unerwarteten Ausnahmen. `json { "error": "Failed to fetch challenges", "details": "Fehlermeldung der Datenbank" }`

Example

```
// Beispiel für einen erfolgreichen Request
const response = await fetch('/api/challenges');
const data = await response.json();
```



```
console.log(data.challenges); // Array der Challenges  
console.log(data.count); // Anzahl der Ergebnisse
```

Function: getListOfChallenges()

getListOfChallenges(): Promise<any[]>

Defined in: [app/api/challenges/list/route.ts:23](#)

Holt eine Liste aller verfügbaren Challenges (Probleme) aus der Datenbank.

Diese Funktion führt eine SQL-Abfrage aus, die alle Einträge aus der Tabelle "Problem" auswählt und nach Titel alphabetisch sortiert.

Returns

Promise<any[]>

Ein Array mit den Datenbankzeilen (Objekten), die die Challenges repräsentieren. Jede Zeile enthält mindestens die Spalten der "Problem"-Tabelle. Gibt ein leeres Array zurück, wenn keine Einträge vorhanden sind.

Throws

Wirft einen Fehler, wenn die Datenbankverbindung fehlschlägt oder die Abfrage syntaktisch ungültig ist. Der Fehler wird nicht abgefangen und muss vom Aufrufer (z.B. dem API-Handler) behandelt werden.

Example

```
const challenges = await getListOfChallenges();
console.log(`Found ${challenges.length} challenges.`);
console.log(challenges[0]?.title); // Titel des ersten Challenges
```

[webapp](#)

[webapp](#) / app/api/challenges/delete/route

app/api/challenges/delete/route

Functions

- [DELETE](#)
- [deleteChallenge](#)

Function: DELETE()

```
DELETE(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ message: string; success: boolean; }>>
```

Defined in: [app/api/challenges/delete/route.ts:74](#)

HTTP DELETE-Handler zum Entfernen eines Challenges.

Dieser Endpunkt erwartet eine JSON mit der `challenge_id`. Er validiert die Eingabe, ruft die Löschfunktion auf und gibt entsprechende Statuscodes zurück (200, 400, 404, 500).

Parameters

request

Request

Das HTTP-Request-Objekt, das den JSON-Body mit der `challenge_id` enthalten muss.

Returns

```
Promise<NextResponse<{ error: string; }> | NextResponse<{ message:  
string; success: boolean; }>>
```

Ein `NextResponse`-Objekt mit folgendem Inhalt:

- **200 OK**: Bei erfolgreicher Löschung. { success: true, message: "Challenge with ID X successfully deleted" }
- **400 Bad Request**: Wenn `challenge_id` fehlt oder ungültig ist. { error: "Challenge ist nicht vorhanden" }
- **404 Not Found**: Wenn die ID in der Datenbank nicht existiert. { error: "Challenge with ID X nicht gefunden" }
- **500 Internal Server Error**: Bei Datenbankfehlern oder unerwarteten

```
Ausnahmen. { error: "Failed to delete challenge", details:  
error.message }
```

Example

```
// Beispiel für einen erfolgreichen Request  
const response = await fetch('/api/challenges', {  
  method: 'DELETE',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ challenge_id: 123 })  
});  
const data = await response.json();  
console.log(data); // { success: true, message: "Challenge with ID 123 su
```

Function: deleteChallenge()

deleteChallenge(id): Promise<boolean>

Defined in: [app/api/challenges/delete/route.ts:28](#)

Löscht ein Problem (Challenge) aus der Datenbank basierend auf seiner ID.

Diese Funktion führt einen SQL-DELETE-Befehl aus und überprüft, ob tatsächlich eine Zeile betroffen war. Sie wirft einen Fehler, wenn die Datenbankabfrage selbst fehlschlägt (z.B. Verbindungsproblem).

Parameters

id

number

Die eindeutige ID des Problems (`problem_id`), das gelöscht werden soll.

Returns

Promise<boolean>

`true`, wenn ein Datensatz mit der angegebenen ID gefunden und gelöscht wurde.
`false`, wenn kein Datensatz mit dieser ID existierte (aber keine Datenbankfehler auftraten).

Throws

Wirft einen Fehler, wenn die Datenbankverbindung unterbrochen wird oder die SQL-Abfrage syntaktisch/logisch fehlschlägt. Dieser Fehler muss vom Aufrufer (z.B. dem API-Handler) abgefangen werden.

Example

```
const success = await deleteChallenge(123);  
if (success) {  
  console.log("Challenge gelöscht.");  
} else {  
  console.log("Challenge nicht gefunden.");  
}
```

[webapp](#)

[webapp](#) / app/api/sandbox/add/route

app/api/sandbox/add/route

Functions

- [POST](#)

Function: POST()

```
POST(request): Promise<NextResponse<{ error: string;  
  required: string[]; }> | NextResponse<{ projectId: any; success:  
  boolean; }> | NextResponse<{ details: any; error: string; }>>
```

Defined in: [app/api/sandbox/add/route.ts:123](#)

Handler für HTTP POST-Anfragen zum Erstellen eines neuen Projekts.

Validiert die Eingabedaten und delegiert die eigentliche Erstellung an `addProject()`.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

```
Promise<NextResponse<{ error: string; required: string[]; }> |  
NextResponse<{ projectId: any; success: boolean; }> | NextResponse<{  
  details: any; error: string; }>>
```

Ein JSON-Antwortobjekt mit Erfolgsmeldung und Projekt-ID oder einem Fehlerstatus.

Example

```
// Erwartetes Body-Format:  
{  
  "title": "Neues Projekt",
```

```
"state": "active",  
"members": [{ "name": "Max Mustermann", "role": "Lead" }]  
}
```

[webapp](#)

[webapp](#) / app/api/sandbox/update/route

app/api/sandbox/update/route

Functions

- [PATCH](#)

Function: PATCH()

```
PATCH(request): Promise<NextResponse<{ error: string; }> |  
NextResponse<{ projectId: any; success: boolean; }>>
```

Defined in: [app/api/sandbox/update/route.ts:186](#)

Handler für HTTP PATCH-Anfragen zum Aktualisieren eines bestehenden Projekts.

Validiert die Anwesenheit der `project_id`, ruft die Update-Funktion auf und gibt das Ergebnis als JSON zurück.

Parameters

request

Request

Die eingehende HTTP-Anfrage.

Returns

```
Promise<NextResponse<{ error: string; }> | NextResponse<{ projectId:  
any; success: boolean; }>>
```

Ein JSON-Antwortobjekt: - 200: Erfolg mit `projectId` - 400: Fehlende `project_id` - 404: Projekt nicht gefunden - 500: Interner Serverfehler

Example

```
// Erwartetes Body-Format:  
{  
  "project_id": 123,
```

```
"title": "Neuer Titel",  
"members": [{ "name": "Anna Schmidt", "role": "Developer" }]  
}
```

[webapp](#)

[webapp](#) / app/api/sandbox/list/route

app/api/sandbox/list/route

Functions

- [GET](#)
- [getListOfSandboxProjects](#)

Function: GET()

```
GET(): Promise<NextResponse<{ count: number; projects: any[];
  success: boolean; }> | NextResponse<{ details: any; error:
  string; }>>
```

Defined in: [app/api/sandbox/list/route.ts:84](#)

Handler für HTTP GET-Anfragen zum Abrufen der Sandbox-Projektliste.

Delegiert die Datenbankabfrage an `getListOfSandboxProjects()` und gibt die Ergebnisse als JSON-Antwort zurück.

Returns

```
Promise<NextResponse<{ count: number; projects: any[]; success: boolean;
  }> | NextResponse<{ details: any; error: string; }>>
```

Ein JSON-Antwortobjekt mit: - `success`: Boolean, ob die Anfrage erfolgreich war - `projects`: Array der Projekt-Objekte - `count`: Anzahl der zurückgegebenen Projekte

Throws

Gibt bei Fehlern eine JSON-Antwort mit Status 500 zurück und enthält Fehlerdetails im `details`-Feld.

Example

```
// Beispielantwort bei Erfolg:
{
  "success": true,
  "projects": [
    {
```

```
    "id": 1,  
    "title": "Beispielprojekt",  
    "experts": [  
      { "expert_id": 5, "name": "Max Mustermann", "role": "Lead" }  
    ]  
  }  
],  
  "count": 1  
}
```


Function: getListOfSandboxProjects()

getListOfSandboxProjects(): Promise<any[]>

Defined in: [app/api/sandbox/list/route.ts:21](#)

Ruft eine Liste von Sandbox-Projekten aus der Datenbank ab.

Diese Funktion führt eine komplexe Abfrage durch, die: 1. Alle Projekte aus der Tabelle "Project" selektiert 2. Über "ProjectRelation" mit Experten verknüpft (LEFT JOIN) 3. Experteninformationen als JSON-Array aggregiert 4. Nach dem letzten Update-Datum absteigend sortiert

Returns

Promise<any[]>

Ein Array von Projekt-Objekten, wobei jedes Projekt ein Feld `experts` enthält, das ein Array von Experten mit `expert_id`, `name` und `role` ist.

Remarks

- Verwendet COALESCE und FILTER um leere Experten-Arrays als [] zurückzugeben
- DISTINCT verhindert Duplikate bei Mehrfachverknüpfungen
- Die Abfrage ist auf 50 Ergebnisse limitiert für Performance

[webapp](#)

[webapp](#) / app/sandbox/add/page

app/sandbox/add/page

Functions

- [default](#)

Function: default()

default(): Element

Defined in: [app/sandbox/add/page.tsx:17](#)

Eine Next.js-Client-Komponente, die eine Eingabemaske zum Erstellen neuer Projekte bietet. Ermöglicht die Pflege von Metadaten sowie die dynamische Zuordnung von Experten über ein RACI-Rollenmodell. *

Returns

Element

Ein gerendertes Formular zur Projekterstellung.

[webapp](#)

[webapp](#) / app/sandbox/project/details/page

app/sandbox/project/details/page

Functions

- [default](#)

Function: default()

default(): Element

Defined in: [app/sandbox/project/details/page.tsx:30](#)

Eine Next.js-Client-Komponente, die eine Detailansicht zur Bearbeitung eines bestehenden Projekts bereitstellt. Lädt Projektdaten sowie die globale Expertenliste parallel, erlaubt die Modifikation aller Felder inklusive RACI-Teamzuweisungen (mit Duplikatsprüfung) und speichert die Änderungen via PATCH-Request. *

Returns

Element

Die gerenderte Detail- und Bearbeitungsseite des Projekts.

[webapp](#)

[webapp](#) / app/sandbox/page

app/sandbox/page

Functions

- [default](#)

Function: default()

default(): Element

Defined in: [app/sandbox/page.tsx:21](#)

Eine Next.js-Client-Komponente, die eine tabellarische Übersicht oder Liste aller Sandbox-Projekte aus der Datenbank anzeigt. Sie lädt die Projektdaten asynchron und bereitet diese direkt für die Anzeige auf. *

Returns

Element

Die gerenderte Übersicht der Sandbox-Projekte.

[webapp](#)

[webapp](#) / app/challenge/add/page

app/challenge/add/page

Functions

- [default](#)

Function: default()

default(): Element

Defined in: [app/challenge/add/page.tsx:28](#)

Eine Next.js-Client-Komponente, die ein Formular zum Erstellen einer neuen Herausforderung (Challenge) bereitstellt. *

Returns

Element

Ein gerendertes React-Formularlement.

[webapp](#)

[webapp](#) / app/challenge/page

app/challenge/page

Functions

- [default](#)

Function: default()

default(): Element

Defined in: [app/challenge/page.tsx:19](#)

Eine Next.js-Client-Komponente, die eine Übersicht aller vorhandenen Probleme darstellt, Daten live von der API lädt und das Löschen sowie die Detailansicht verwaltet.

*

Returns

Element

Ein gerendertes UI-Element für die Problem-Verwaltung.

[webapp](#)

[webapp](#) / app/layout

app/layout

Variables

- [metadata](#)

Functions

- [default](#)

[webapp](#)

[webapp](#) / [app/layout](#) / metadata

Variable: metadata

```
const metadata: Metadata
```

Defined in: [app/layout.tsx:11](#)

[webapp](#)

[webapp](#) / [app/layout](#) / default

Function: default()

default(__namedParameters): Element

Defined in: [app/layout.tsx:18](#)

Parameters

__namedParameters

children

ReactNode

Returns

Element

[webapp](#)

[webapp](#) / app/components/Agentbar

app/components/Agentbar

Functions

- [Agentbar](#)

Function: Agentbar()

Agentbar(): Element

Defined in: [app/components/Agentbar.tsx:10](#)

Die seitliche Ansicht auf dem der KI-Agent platziert wird. Hier sollen die Befehle an den Agent übernommen werden.

Returns

Element

[webapp](#)

[webapp](#) / app/components/Sidebar

app/components/Sidebar

Functions

- [Sidebar](#)

Function: Sidebar()

Sidebar(): Element

Defined in: [app/components/Sidebar.tsx:12](#)

Die Navigationsleiste mit Verbindung zu den Bereichen Expertennetzwerk, Challenges und Sandbox-Projekte.

Returns

Element

[webapp](#)

[webapp](#) / app/components/Topbar

app/components/Topbar

Functions

- [Topbar](#)

Function: Topbar()

Topbar(): Element

Defined in: [app/components/Topbar.tsx:14](#)

Die Topbar, die auf allen Seiten sichtbar ist. Sie enthält eine Suchleiste für die schnelle Suche nach Expertisen, Problemen oder Projekten. Außerdem gibt es zwei prominente Buttons: Einen zum Hinzufügen eines neuen Problems (Challenge) und einen zum Hinzufügen eines neuen Projekts (Sandbox).

Returns

Element

[webapp](#)

[webapp](#) / app/search/page

app/search/page

Functions

- [default](#)

Function: default()

default(): Element

Defined in: [app/search/page.tsx:38](#)

Eine Next.js-Client-Komponente, die eine globale, interaktive Suchoberfläche bereitstellt. Durchsucht parallel Challenges, Projekte und Expert*innen anhand einer einzigen Benutzereingabe (Query) in Echtzeit auf Client-Eite. *

Returns

Element

Das gerenderte Suchfenster inklusive Suchergebnissen.

[webapp](#)

[webapp](#) / app/page

app/page

Functions

- [default](#)

[webapp](#)

[webapp](#) / [app/page](#) / default

Function: default()

default(): Element

Defined in: [app/page.tsx:5](#)

Returns

Element

[webapp](#)

[webapp](#) / app/relations/add/page

app/relations/add/page

Functions

- [default](#)

Function: default()

default(props): Element

Defined in: [app/relations/add/page.tsx:15](#)

Eine Next.js-Client-Komponente, die ein Formular zum Erstellen und Speichern eines neuen Experten-Profiles bereitstellt. Die Daten werden sowohl an die Datenbank übertragen als auch über Callbacks an die übergeordnete Komponente zurückgegeben.

*

Parameters

props

[AddExpertViewProps](#)

Die Props für die Komponente.

Returns

Element

Ein gerendertes UI-Formular für die Expertenerfassung.

[webapp](#)

[webapp](#) / app/relations/update/page

app/relations/update/page

Functions

- [default](#)

Function: default()

default(props): Element

Defined in: [app/relations/update/page.tsx:15](#)

Eine Next.js-Client-Komponente, die als Formular zum Bearbeiten oder Erstellen eines Experten-Profiles dient. Wenn ein `expertId`-Query-Parameter in der URL existiert, schaltet die Ansicht in den **Bearbeitungsmodus** und lädt die bestehenden Daten vorab aus der Datenbank. *

Parameters

props

[AddExpertViewProps](#)

Die Props für die Komponente.

Returns

Element

Ein gerendertes UI-Formular zur Expert*innen-Verwaltung.

[webapp](#)

[webapp](#) / app/relations/page

app/relations/page

Functions

- [default](#)

[webapp](#)

[webapp](#) / [app/relations/page](#) / default

Function: default()

default(): Element

Defined in: [app/relations/page.tsx:34](#)

Returns

Element

[webapp](#)

[webapp](#) / app/relations/Person

app/relations/Person

Interfaces

- [Expert](#)
- [ExpertFormData](#)
- [Organization](#)

Variables

- [getUserFromDB](#)
- [initialExperts](#)

[webapp](#)

[webapp](#) / [app/relations/Person](#) / getUserFromDB

Variable: getUserFromDB

```
const getUserFromDB: () => Promise<Expert[]>
```

Defined in: [app/relations/Person.tsx:48](#)

Returns

Promise<[Expert](#)[]>

[webapp](#)

[webapp](#) / [app/relations/Person](#) / initialExperts

Variable: initialExperts

```
const initialExperts: Expert[]
```

Defined in: [app/relations/Person.tsx:85](#)

[webapp](#)

[webapp](#) / [app/relations/Person](#) / Expert

Interface: Expert

Defined in: [app/relations/Person.tsx:12](#)

Properties

description?

optional **description?**: string

Defined in: [app/relations/Person.tsx:22](#)

economic

economic: boolean

Defined in: [app/relations/Person.tsx:25](#)

email?

optional **email?**: string

Defined in: [app/relations/Person.tsx:20](#)

expert_fields

expert_fields: string[]

Defined in: [app/relations/Person.tsx:24](#)

expert_id

expert_id: number

Defined in: [app/relations/Person.tsx:13](#)

last_contact?

optional **last_contact?:** string

Defined in: [app/relations/Person.tsx:23](#)

name

name: string

Defined in: [app/relations/Person.tsx:14](#)

organization?

optional **organization?:** [Organization](#)

Defined in: [app/relations/Person.tsx:28](#)

other_organizations

other_organizations: string[]

Defined in: [app/relations/Person.tsx:18](#)

phone?

optional **phone?:** string

Defined in: [app/relations/Person.tsx:21](#)

firstname

firstname: string

Defined in: [app/relations/Person.tsx:15](#)

primary_organization

primary_organization: string

Defined in: [app/relations/Person.tsx:17](#)

science

science: boolean

Defined in: [app/relations/Person.tsx:26](#)

scientificAreas

scientificAreas: string[]

Defined in: [app/relations/Person.tsx:19](#)

social

social: boolean

Defined in: [app/relations/Person.tsx:27](#)

title?

optional **title?:** string

Defined in: [app/relations/Person.tsx:16](#)

[webapp](#)

[webapp](#) / [app/relations/Person](#) / ExpertFormData

Interface: ExpertFormData

Defined in: [app/relations/Person.tsx:31](#)

Properties

description

description: string

Defined in: [app/relations/Person.tsx:41](#)

economic

economic: boolean

Defined in: [app/relations/Person.tsx:43](#)

email

email: string

Defined in: [app/relations/Person.tsx:38](#)

expert_fields

expert_fields: string

Defined in: [app/relations/Person.tsx:42](#)

last_contact?

optional **last_contact?**: string

Defined in: [app/relations/Person.tsx:40](#)

name

name: string

Defined in: [app/relations/Person.tsx:32](#)

other_organizations

other_organizations: string

Defined in: [app/relations/Person.tsx:36](#)

phone

phone: string

Defined in: [app/relations/Person.tsx:39](#)

prename

prename: string

Defined in: [app/relations/Person.tsx:34](#)

primary_organization

primary_organization: string

Defined in: [app/relations/Person.tsx:35](#)

science

science: boolean

Defined in: [app/relations/Person.tsx:44](#)

scientificAreas

scientificAreas: string

Defined in: [app/relations/Person.tsx:37](#)

social

social: boolean

Defined in: [app/relations/Person.tsx:45](#)

title

title: string

Defined in: [app/relations/Person.tsx:33](#)

[webapp](#)

[webapp](#) / [app/relations/Person](#) / Organization

Interface: Organization

Defined in: [app/relations/Person.tsx:3](#)

Properties

description?

optional **description?**: string

Defined in: [app/relations/Person.tsx:8](#)

field?

optional **field?**: string

Defined in: [app/relations/Person.tsx:7](#)

location?

optional **location?**: string

Defined in: [app/relations/Person.tsx:6](#)

name

name: string

Defined in: [app/relations/Person.tsx:5](#)

organization_id

organization_id: number

Defined in: [app/relations/Person.tsx:4](#)

BSD 3-Clause License

Copyright 2007, 2008 The Python Markdown Project (v. 1.7 and later)

Copyright 2004, 2005, 2006 Yuri Takhteyev (v. 0.2-1.6b)

Copyright 2004 Manfred Stienstra (the original version)

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.